Wallet Service

RecargaPay – Technical Assessment

Java 25 · Spring Boot 4 · PostgreSQL 17 · Redis 7 · Docker

Project Overview

Core Purpose

API REST mission-critical para carteiras digitais: depósito, saque, transferência, saldo atual e histórico.

Stack

Java 25 · Spring Boot 4 · PostgreSQL 17 · Redis 7 · Flyway · Testcontainers · Springdoc OpenAPI 3.

Arquitetura

Monolito modular com fronteira explícita wallet ↔ transaction, preparado para extração a microserviços.

Arquitetura — Monolito Modular

Wallet Module

- WalletController
- WalletService
- WalletRepository
- Wallet (entity)
- DTOs de request/response

Transaction Module

- TransactionController
- TransactionService
- WalletComponent ⬡
- TransactionRepository
- Transaction (entity)

Shared Module

- DistributedLockService
- CacheConfig
- GlobalExceptionHandler
- Enums / Exceptions

⬡ WalletComponent = proxy de integração. Substituído por Feign/RestClient na migração para microserviços — sem alterar TransactionService.

Lock Distribuído com Redis (DA-02 · DA-03)

Fluxo de Aquisição

1. SET wallet:lock:{id} {uuid} NX PX 10000
   → NX: atômico, só define se não existir
   → PX 10s: TTL protege contra crash

2. Se bloqueado → retry a cada 50ms por 5s
   → Timeout: LockAcquisitionException → HTTP 409

3. Executa a operação dentro do lock

4. Libera via Lua script atômico:
   if get(key)==uuid then del(key) end
   → Só o dono pode liberar

DA-03 — Release após TX Commit

❌ **Sem DA-03:**
lock → modifica → libera → [TX pendente] → outra thread lê saldo errado

✅ **Com DA-03:**
lock → modifica → TX commita → libera → outra thread lê saldo correto

Prevenção de Deadlock

A→B e B→A simultâneos causam deadlock se locks adquiridos fora de ordem.

Solução: adquirir locks sempre em ordem crescente de UUID.

```
sortedIds = [sourceId, destId].stream().sorted().toList()
```

Data Model — PostgreSQL

tb_wallet

- id UUID (PK)
- user_id UUID UNIQUE
- balance DECIMAL(19,4)
- currency VARCHAR(3) 'BRL'
- created_at TIMESTAMPTZ
- updated_at TIMESTAMPTZ
- ——— CHECK (balance >= 0)

tb_transaction

- id UUID (PK)
- wallet_id UUID (FK)
- type DEPOSIT | WITHDRAWAL
- TRANSFER_IN | TRANSFER_OUT
- amount DECIMAL(19,4)
- balance_after DECIMAL(19,4)
- reference_id UUID (nullable)
- description VARCHAR(255)
- created_at TIMESTAMPTZ
- ——— CHECK (amount > 0)
- ——— CHECK (balance_after >= 0)

reference_id: TRANSFER_OUT.id ↔ TRANSFER_IN.id — vínculo bidirecional entre os dois registros de uma transferência.

Saldo Histórico & Cache Redis (DA-04)

Saldo Histórico — O(log n)

DA-04: balance_after desnormalizado em cada transação elimina replay.

Consulta:

```
SELECT balance_after FROM tb_transaction
WHERE wallet_id = :id
  AND created_at <= :ts
ORDER BY created_at DESC
LIMIT 1
```

Índice composto (wallet_id, created_at) garante O(log n).
Se nenhuma transação existe antes do timestamp → retorna saldo zero.

Cache Redis — Wallet Entity

Cache name	wallets
Chave	wallets::{walletId}
TTL	10 minutos
Serialização	JSON + @class (polimórfico)
Null values	desativados
@Cacheable	findById (leitura)
@CachePut	após qualquer escrita
@CacheEvict	após operação de transação

API Design — Endpoints REST

POST	/api/v1/wallets	Cria nova carteira
GET	/api/v1/wallets/{walletId}	Retorna dados da carteira
GET	/api/v1/wallets/{walletId}/balance	Saldo atual
GET	/api/v1/wallets/{walletId}/balance/history?at=	Saldo em ponto no tempo
POST	/api/v1/wallets/{walletId}/deposit	Deposita valor
POST	/api/v1/wallets/{walletId}/withdraw	Saca valor
POST	/api/v1/wallets/transfer	Transfere (atômico, anti-deadlock)
GET	/api/v1/wallets/{walletId}/transactions	Histórico paginado
GET	/actuator/health	Health check

Tratamento de Erros — RFC 7807

404	**`WalletNotFoundException`** Carteira não encontrada	*ProblemDetail*
409	**`WalletAlreadyExistsException`** Usuário já possui carteira	*ProblemDetail*
409	**`LockAcquisitionException`** Sistema sob alta carga — lock não obtido	*ProblemDetail*
422	**`InsufficientFundsException`** Saldo insuficiente para saque/transferência	*ProblemDetail*
400	**`InvalidRequestException`** Regra de negócio violada (ex.: transferência para si mesmo)	*ProblemDetail*
400	**`MethodArgumentNotValidException`** Bean Validation — mapa de erros por campo	*ProblemDetail*
500	**`Exception (fallback)`** Erro inesperado	*ProblemDetail*

Todas as respostas de erro seguem RFC 7807 (ProblemDetail) — type, title, status, detail, instance.

Estratégia de Testes

Unit Tests ~60%

Lógica de serviço · Edge cases · Mockito

Integration ~30%

Testcontainers · PostgreSQL real · Redis real · TX behavior

E2E / API ~10%

Spring Boot Test completo · MockMvc · OpenAPI contract

Cenários cobertos:

criação de carteira · duplicidade · saldo insuficiente · transferência para si mesmo · prevenção de deadlock · saldo histórico (timestamp anterior) · histórico paginado

Configuração por Ambiente & Deploy

Perfil	Banco	Redis	Hikari pool	Log
local	localhost:5432	localhost:6379	padrão	DEBUG
dev	env vars	env vars	padrão	DEBUG
stage	env vars	env vars	5–10	INFO
production	env vars	env vars	10–20	WARN

Quick Start

```
docker-compose up -d
mvn spring-boot:run
# Swagger UI → http://localhost:8080/swagger-ui.html
# Redoc   → http://localhost:8080/redoc.html
```

Decisões Arquiteturais & Trade-offs

DA-01 **Monolito modular**

WalletComponent isola a fronteira — extração para microserviços sem alterar TransactionService.

DA-02 **Lock Redis vs. DB lock**

SET NX PX — funciona em qualquer topologia. SELECT FOR UPDATE quebra em microserviços.

DA-03 **Release após commit**

TransactionSynchronization.afterCompletion elimina janela de inconsistência read-after-write.

DA-04 **balance_after desnorm.**

O(log n) em histórico vs replay O(n). Custo: espaço extra por registro de transação.

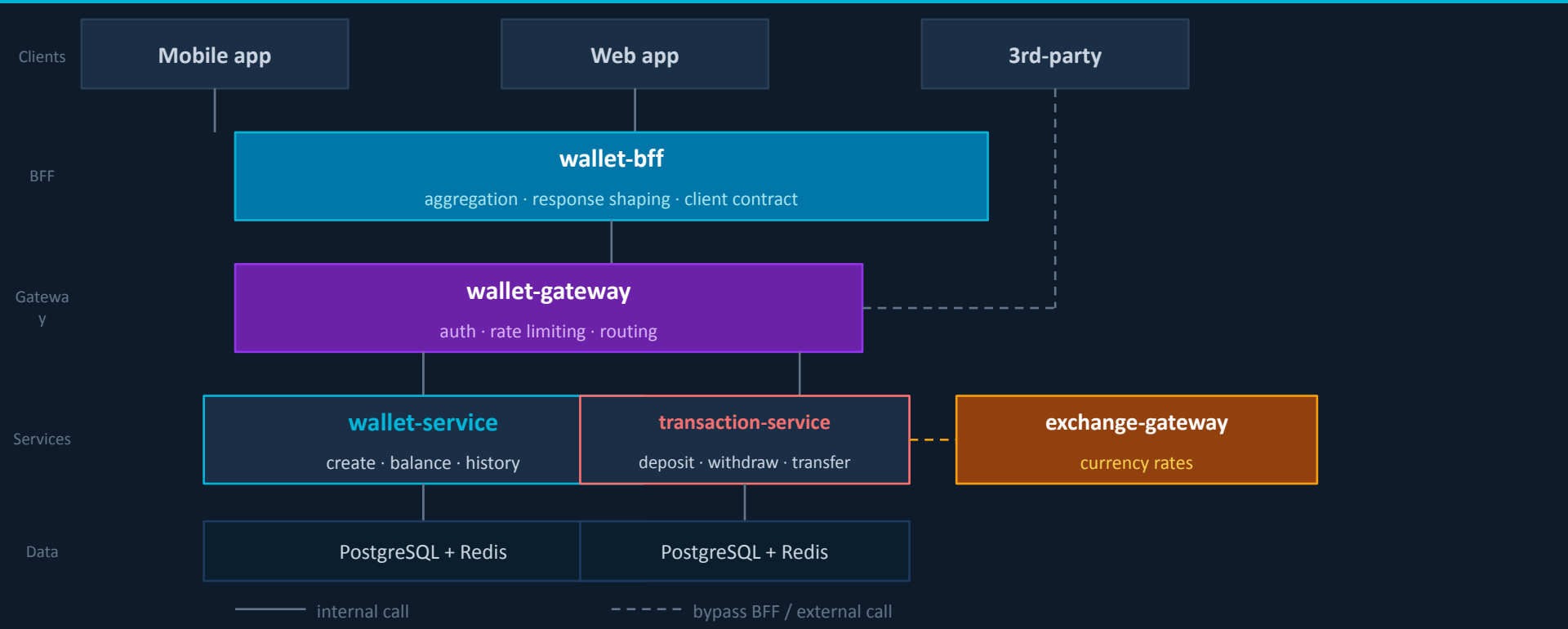
DA-05 **Constraints no banco**

CHECK constraints no schema como segunda linha de defesa — protege contra bugs e acessos diretos.

Scope **Sem auth / rate limit**

OAuth2/JWT e sliding-window rate limiting são passos de produção — fora do escopo do assessment.

Microservices — Proposed Architecture



Ready to Build

Java 25 · Spring Boot 4 · Redis locks · Cache · Testes com Testcontainers · ~7h

Monolito Modular	Lock Redis Distribuído	Cache Redis	Histórico O(log n)	RFC 7807 Errors